Final Project Submission

DeepCompile: AI-Driven Compiler Optimization

Roll Number: 24CSB0A06 · Date: April 19, 2026

DeepCompile User Manual

What DeepCompile Does

DeepCompile reads C/C++ source code, analyzes it, applies safe source-to-source rewrites, verifies the result, runs a security audit, and produces explainability artifacts.

Recommended Mode

For formal and reproducible usage, run in deterministic rule-based mode:

```powershell
python run_pipeline.py --no-llm
```

Common Tasks

Run the orchestrated workflow:

```powershell
python run_pipeline.py .\MicroBenchmarks\Testcases\TC11_buffer_overflow_loop.cpp --no-llm
```

Run the legacy baseline pipeline:

```powershell
python run_pipeline.py .\MicroBenchmarks\Testcases\TC11_buffer_overflow_loop.cpp
--basic-pipeline --no-llm
```

Export a Markdown explainability report:

```powershell
```

python run_pipeline.py .\MicroBenchmarks\Testcases\TC11_buffer_overflow_loop.cpp --no-llm --export-md .\logs\report.md

```

Export an HTML explainability report:

```powershell

python run_pipeline.py .\MicroBenchmarks\Testcases\TC11_buffer_overflow_loop.cpp --no-llm --export-html .\logs\report.html

```

Output You Should Expect

- a pipeline summary in the terminal
- an optimized source file under `MicroBenchmarks/Generated_optimisation/`
- optional Markdown/HTML explainability reports
- workflow metadata including stop reason and iteration count when orchestration is active

Safety Notes

- if verification fails, the workflow can roll back
- if security detects a new high-severity issue, the workflow can roll back
- if retries stop improving the result, the workflow escalates for review
```